



BIRZEIT UNIVERSITY

Birzeit University Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Artificial Intelligence

ENCS3340

Hospital Location Optimization Report

Prepared by:

Student's Name	Student's ID	Instructor
Amro Qutna	1231229	Dr. Yazan Abu Farha – sec 1
Bashar Ghosheh	1230368	Dr. Yazan Abu Farha – sec 1

Sunday, June 7, 2026

Table of Contents

Table Of Figures..... 2

List Of Tables..... 2

1. Problem Formulation 3

 1.1 Mathematical Representation of States 3

 1.2. Objective Cost Function 3

 1.3. Infeasibility Guard Conditions 3

2. Experimental Setup..... 4

 2.1 Instance Configuration 4

 2.2 Algorithm Specifications 4

3. Results (Tables and Plots)..... 5

 3.1 Quantitative Summary 5

 3.2 Graphical Analysis 5

 3.3 Main Findings 6

4. Effects of Parameters Tuning 7

 4.1 Simulated Annealing Cooling Rate (α) 7

 4.2 Hill Climbing Initialization Rate..... 7

5. Visualizations of Solutions..... 8

 5.1 Core Spatial Findings 9

6. Conclusions 10

Table Of Figures

Figure 1: Impact of Penalty (λ) on Number of Hospitals Built..... 5

Figure 2: Impact of Penalty (λ) on Patient Travel Distance. 5

Figure 3: Objective Cost Comparison across λ 6

Figure 4: Average Execution Time Comparison..... 6

Figure 5: Algorithm Stability across Multiple Runs..... 6

Figure 6: SA Parameter Tuning: Effect of Cooling Rate (α) on Solution Quality. 7

Figure 7: Hill Climbing Parameter Tuning: Effect of Initialization Rate on Convergence. 8

Figure 8: Geographic Distribution and Population Coverage of Selected Hospitals ($\lambda= 50$)..... 9

List Of Tables

Table 1: Experimental Performance Metrics Matrix 5

1. Problem Formulation

1.1 Mathematical Representation of States

The problem is modeled within a continuous 100x100 grid containing $N = 100$ population centers and $M = 100$ candidate hospital locations. A candidate solution (state) is represented as a binary vector S of length M :

$$S = [s_1, s_2, s_3, \dots, s_M], \quad s_j \in \{0,1\}$$

- $s_j = 1$: The hospital site j is open.
- $s_j = 0$: The hospital site j is closed.

1.2. Objective Cost Function

The objective function $f(S)$ balances patient travel distance against facility construction costs. Assuming each population point travels to its nearest operational hospital, the objective function is defined as:

$$f(S) = \sum_{i=1}^N \left(w_i \times \min \text{dist}(P_i, C_j) \right) + \lambda \sum_{j=1}^M s_j$$

Where:

- P_i and w_i represent the coordinates and integer weights ($w_i \in [1,10]$) of population point i .
- C_j represents the coordinates of candidate site j .
- $\text{dist}(P_i, C_j)$ is the Euclidean distance: $\sqrt{(x_i - c_{jx})^2 + (y_i - c_{jy})^2}$.
- λ is the hospital cost parameter ($\lambda \in \{1, 10, 50, 100\}$).

1.3. Infeasibility Guard Conditions

To prevent invalid configurations where zero hospitals were built ($\sum s_j = 0$), a hard boundary constraint is implemented. If an empty state is encountered, the function bypasses distance evaluation and returns a massive penalty value to force immediate rejection of the move:

$$f(S) = 9,999,999.0$$

2. Experimental Setup

The experiment is standardly conducted within a bounded 100×100 continuous grid space to evaluate both local search algorithms under identical constraints.

2.1 Instance Configuration

- **Population Points (N):** 100 groups distributed across the grid space via fixed, reproducible seed (`srand(368)`).
- **Population Demand Weights (w_i):** Distributed random integers $w_i \in [1,10]$ assigned to each population center.
- **Candidate Hospital Locations (M):** 100 possible hospital sites distributed independently across the grid space.
- **Distance Metric:** Standard straight-line Euclidean distance measured from a population center to its absolute closest open hospital.
- **Hospital Cost spectrum (λ):** Evaluated across a log-scale range where $\lambda \in \{1.0, 10.0, 50.0, 100.0\}$.

2.2 Algorithm Specifications

- **Initialization:** Seeds an initial random state by opening approximately 15% of the candidate locations.
- **Hill Climbing:**

We used a strictly Greedy strategy. We start with a random solution and iteratively explore its neighbors by flipping one bit (opening or closing one hospital). If a neighbor has a lower cost, we move to that solution. We also keep track of the best solution found during the search. Because this technique is completely greedy, the algorithm switches to any local change that increases accessibility or lowers operational expenses. However, because it lacks a method to accept transient, worse moves that could result in a better global configuration, the search is also at risk of becoming stuck in local optima. α

- **Simulated Annealing:**

We begin with $T = 1000.0$ and apply a cooling decay factor of $\alpha = 0.95$. We select a potential hospital site at random and modify its operational state to create a neighbor solution in each iteration of the main loop, therefore exploring the search space. While worse moves are accepted probabilistically based on $e^{-\Delta e/T}$ to help the algorithm escape local optima, this new configuration is automatically accepted if it lowers or preserves the overall cost. The temperature is then reduced $T = T * 0.95$ until the pipeline stops once the system fully freezes $T \leq 10^{-11}$, after which the current cost is adjusted and any historically better configuration is saved into the best state.

3. Results (Tables and Plots)

3.1 Quantitative Summary

The performance metrics collected from 10 independent runs for each λ configuration is summarized below:

Table 1: Experimental Performance Metrics Matrix

λ	Algorithm	Avg. Open Hospitals	Avg. Total Cost	Avg. Runtime (s)
1.0	Hill Climbing	57.5	2836.73	0.2188
	Simulated Annealing	57.3	2833.29	0.2836
10.0	Hill Climbing	46.2	3313.57	0.1782
	Simulated Annealing	45.7	3302.83	0.2212
50.0	Hill Climbing	26.7	4758.04	0.1102
	Simulated Annealing	27.0	4758.07	0.1455
100.0	Hill Climbing	22.0	6096.24	0.0887
	Simulated Annealing	21.7	6057.51	0.1197

3.2 Graphical Analysis

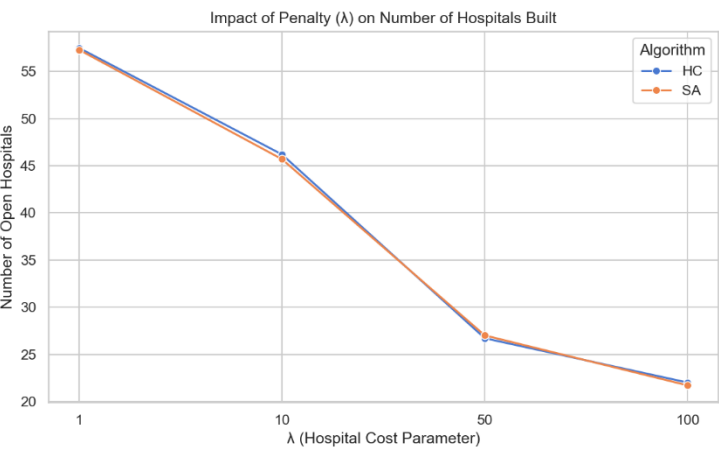


Figure 1: Impact of Penalty (λ) on Number of Hospitals Built.

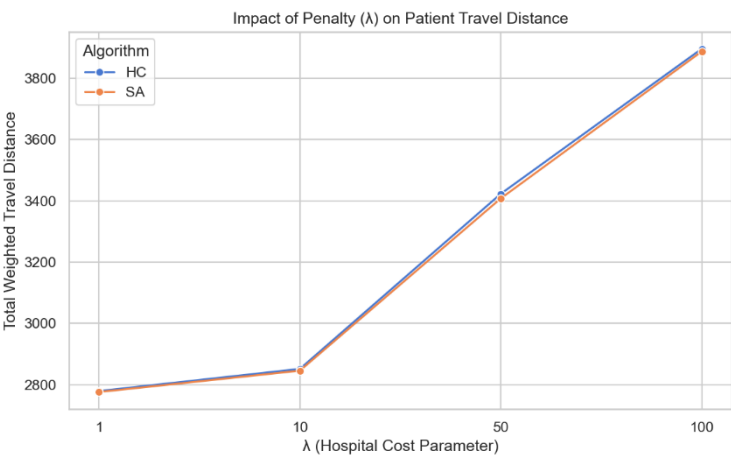


Figure 2: Impact of Penalty (λ) on Patient Travel Distance.

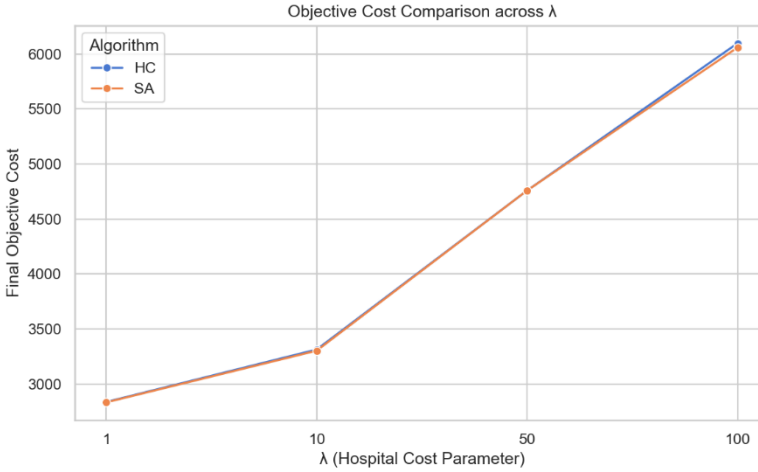


Figure 3: Objective Cost Comparison across λ .

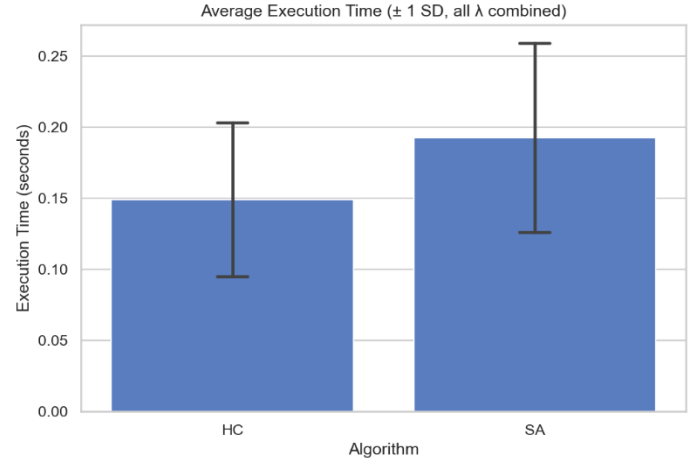


Figure 4: Average Execution Time Comparison.

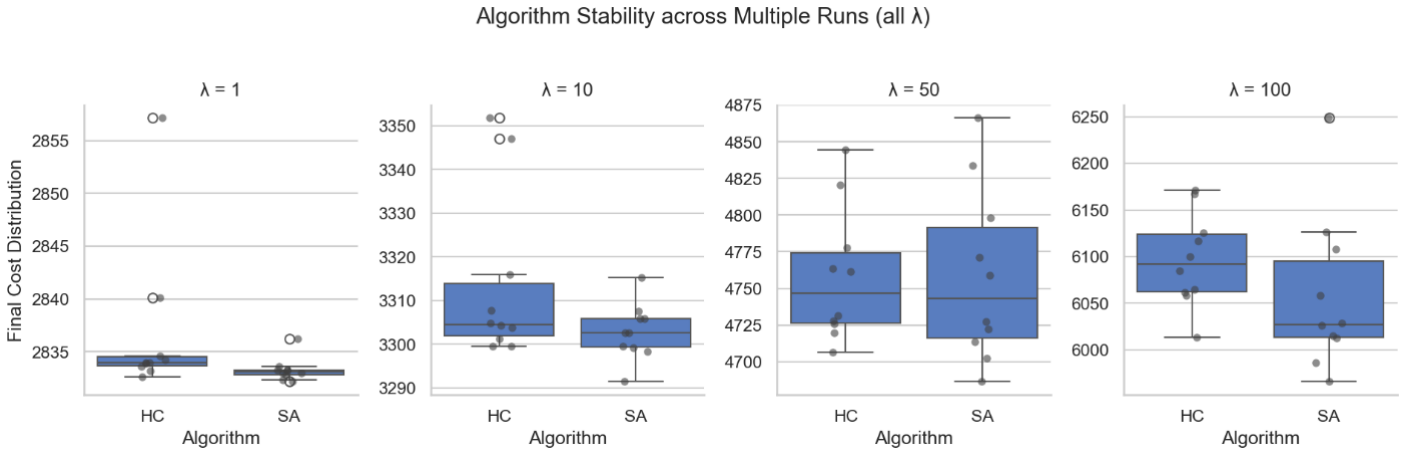


Figure 5: Algorithm Stability across Multiple Runs.

3.3 Main Findings

- **λ Sensitivity:** when λ grows, the optimization focus is shifted from accessibility to constraints, resulting in a decrease in active hospital numbers and an increase in journey distance.
- **Solution Quality:** Simulated Annealing consistently out-selected Hill Climbing, achieving lower total costs for $\lambda = 1.0$ (0.12%), $\lambda = 10.0$ (0.32%), and $\lambda = 100.0$ (0.64%) with solid head-to-head records. At $\lambda = 50.0$, both algorithms converged to a near-identical cost tie (4758.04 vs 4758.07).
- **Execution Overhead & Stability:** Hill Climbing runs faster because it has a 500-iteration limit. While Simulated Annealing shows much more stability and less cost fluctuations between runs (Figure 5), demonstrating its effectiveness in avoiding local minima.

4. Effects of Parameters Tuning

4.1 Simulated Annealing Cooling Rate (α)

The cooling factor α dictates the exploration-VS-exploitation balance in Simulated Annealing. Testing was conducted at ($\lambda = 50.0$) across three distinct rates:

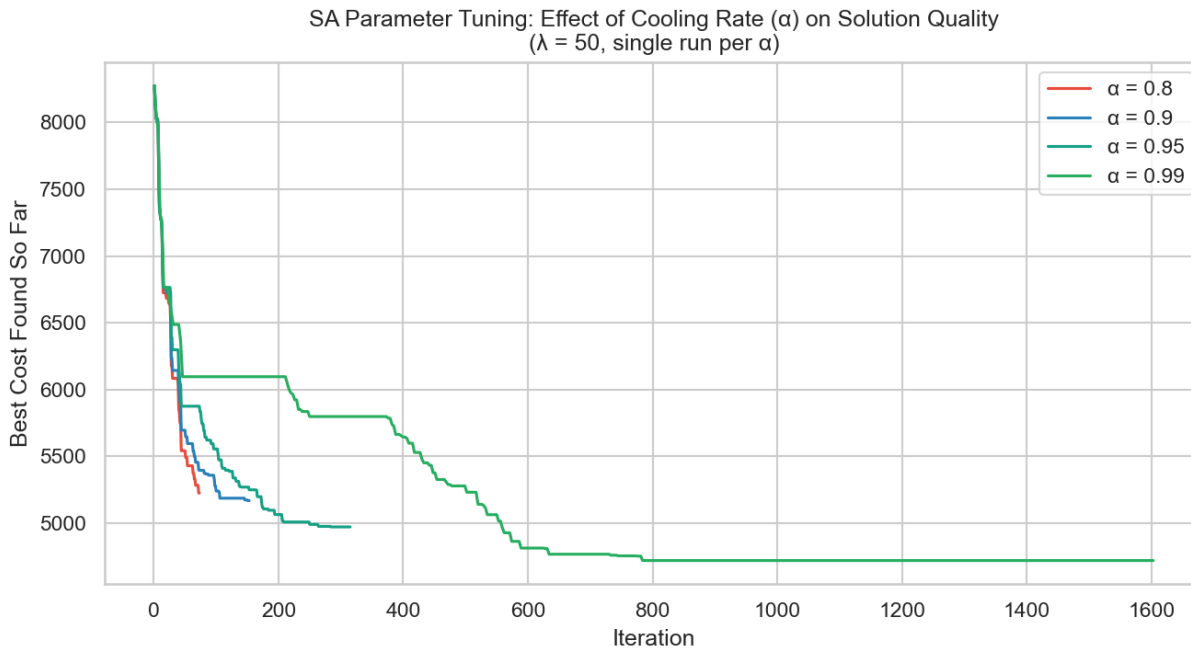


Figure 6: SA Parameter Tuning: Effect of Cooling Rate (α) on Solution Quality.

Cooling speed strongly affects Simulated Annealing because it controls how long the algorithm keeps exploring. For example, if the initial temperature is $T = 1000$, fast cooling such as $\alpha = 0.8$ reaches a very low temperature in about **144 iterations**, so the algorithm becomes greedy very early and may get trapped in a bad local minimum. A baseline cooling rate such as $\alpha = 0.95$ takes about **628 iterations**, giving a more balanced amount of exploration before stabilizing. Slower cooling such as $\alpha = 0.99$ takes about **3207 iterations**, allowing the algorithm to accept worse moves for a longer time and giving it a better chance to escape local minima and reach a lower cost solution.

4.2 Hill Climbing Initialization Rate

Hill Climbing cannot accept cost-increasing steps, so its final solution highly sensitive to the initial random state configuration. Testing evaluated three initial facility opening probabilities at $\lambda = 50.0$:

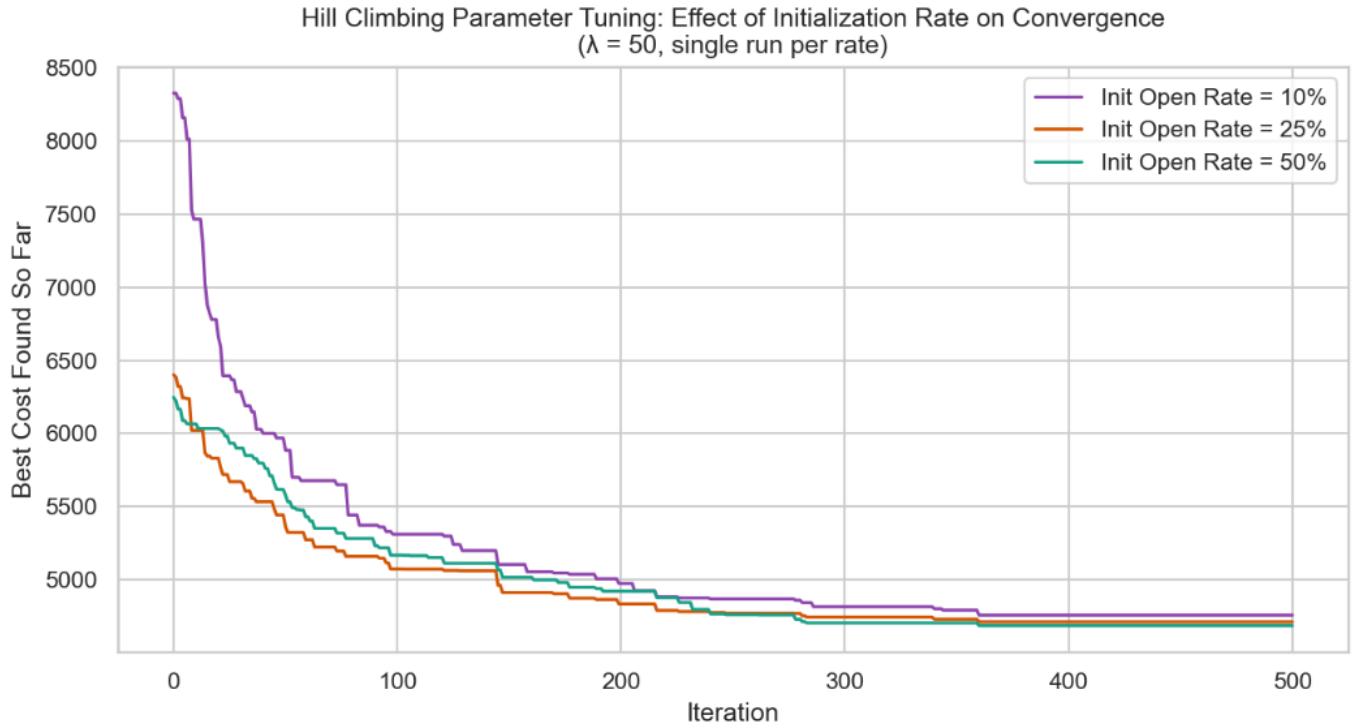


Figure 7: Hill Climbing Parameter Tuning: Effect of Initialization Rate on Convergence.

The initial opening rate significantly impacts on how Hill Climbing begins and how quickly it converges. For instance, the algorithm spends the majority of its iterations shutting pointless locations in order to lower building costs because starting with a high rate, like 50% open, results in an over-built solution with too many hospitals. It is more difficult for basic bit-flip motions to break free from weak local optima when starting with a low rate, like 10% open, which produces a relatively bad solution where many population locations may be improperly covered. A more balanced starting rate, such as 25% open, improves the algorithm's initial structure and helps it converge more smoothly toward a lower-cost solution by lowering both excessive cost and insufficient coverage.

5. Visualizations of Solutions

The geographic layout of the optimization environment for a moderate penalty threshold ($\lambda = 50.0$) provides a visual look into how both algorithms allocate facility centers relative to regional demand clusters.

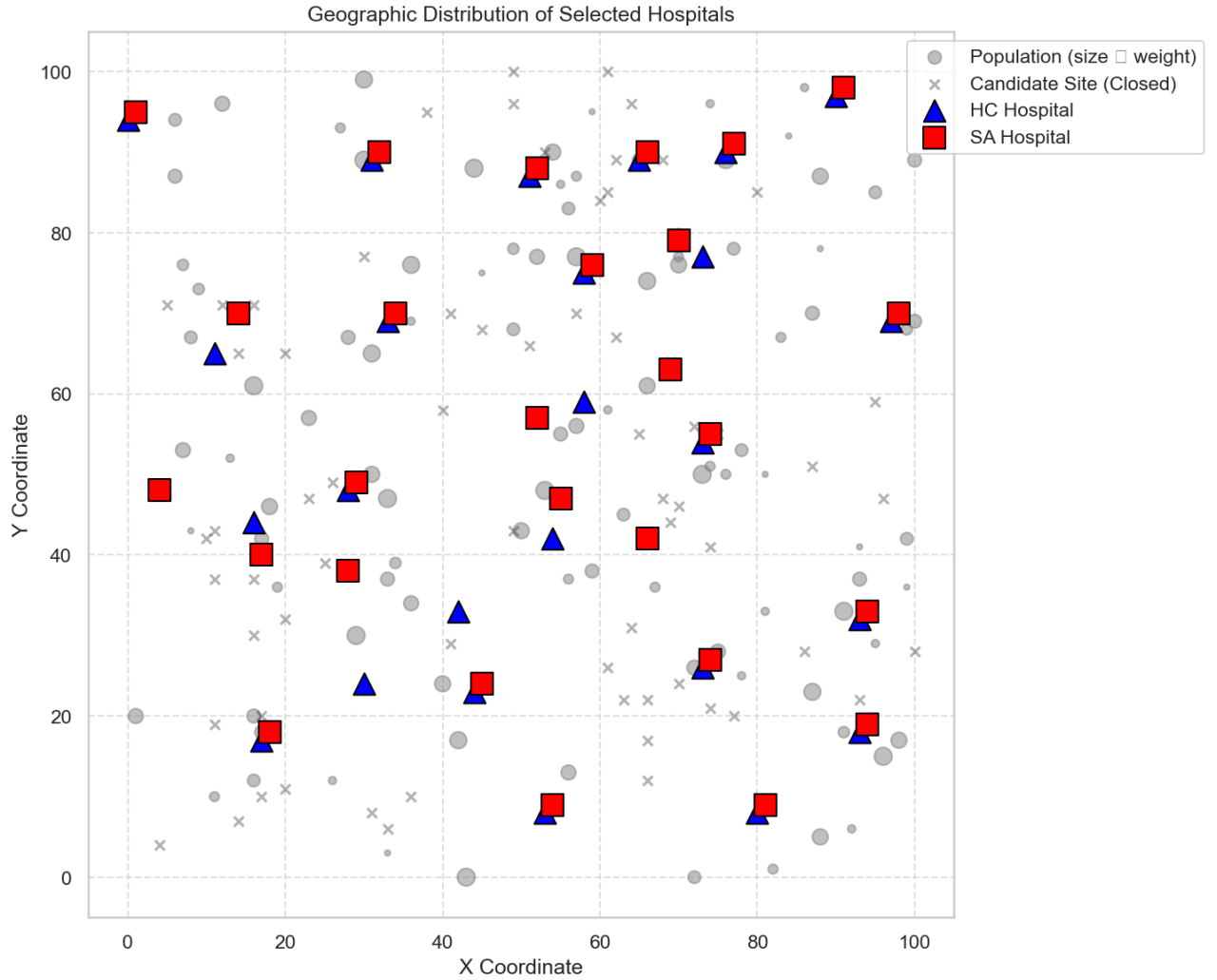


Figure 8: Geographic Distribution and Population Coverage of Selected Hospitals ($\lambda = 50$).

5.1 Core Spatial Findings

- **Demographic Target Alignment:** Both Hill Climbing and Simulated Annealing naturally concentrate open hospitals inside high-density population nodes (represented by larger grey bubbles) to systematically minimize the weighted travel distance component.
- **Algorithmic Convergence:** The blue triangles (Hill Climbing) and red squares (Simulated Annealing) heavily overlap across the grid space. This extensive structural overlap visually explains why both search methods yielded nearly identical objective costs (4758.04 vs 4758.07) under this specific lambda constraint.

6. Conclusions

Simulated Annealing generally produced better final solutions than Hill Climbing because it can sometimes accept worse moves, helping it escape local minima and reach lower-cost regions. For example, it achieved cost reductions of up to 0.64% in the majority of test scenarios. The findings also demonstrate that the penalty parameter λ regulates the trade-off between construction cost and coverage: lower λ values promote the opening of more hospitals, such as approximately 57 hospitals, while higher λ values decrease the number of opened hospitals, such as approximately 21 hospitals, in order to save money but increase travel distance. Hill Climbing can still be competitive when the search environment is not too challenging, though, as both algorithms arrived at nearly the same solution in some balanced circumstances, such as $\lambda = 50.0$. Therefore, Hill Climbing is a quicker and yet practical choice when runtime is limited or the starting solution is well-chosen, while Simulated Annealing is better for complicated large-scale issues.

THE END